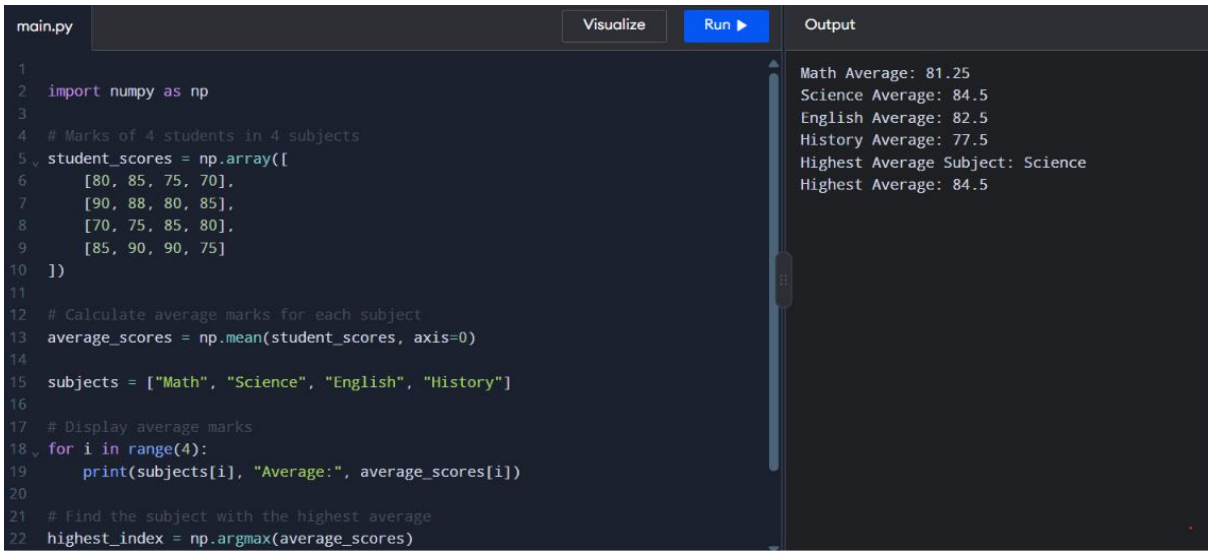
1. **Scenario:** You are working on a project that involves analyzing student performance data for a class of 32 students. The data is stored in a NumPy array named `student_scores`, where each row represents a student and each column represents a different subject. The subjects are arranged in the following order: Math, Science, English, and History. Your task is to calculate the average score for each subject and identify the subject with the highest average score.

Question: How would you use NumPy arrays to calculate the average score for each subject and determine the subject with the highest average score? Assume 4x4 matrix that stores marks of each student in given order.




```
import numpy as np

# Marks of 4 students in 4 subjects
student_scores = np.array([
    [80, 85, 75, 70],
    [90, 88, 80, 85],
    [70, 75, 85, 80],
    [85, 90, 90, 75]
])

# Calculate average marks for each subject
average_scores = np.mean(student_scores, axis=0)

subjects = ["Math", "Science", "English", "History"]
```

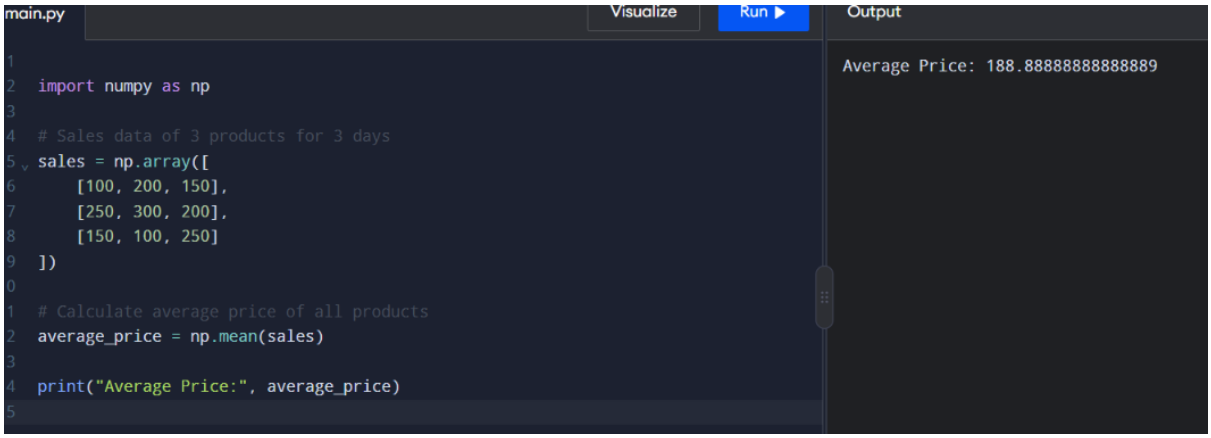
```
# Display average marks
for i in range(4):
    print(subjects[i], "Average:", average_scores[i])

# Find the subject with the highest average
highest_index = np.argmax(average_scores)

print("Highest Average Subject:", subjects[highest_index])
print("Highest Average:", average_scores[highest_index])
```

2. **Scenario:** You are a data analyst working for a company that sells products online. You have been tasked with analyzing the sales data for the past month. The data is stored in a NumPy array.

Question: How would you find the average price of all the products sold in the past month? Assume 3x3 matrix with each row representing the sales for a different product




```
import numpy as np

# Sales data of 3 products for 3 days
sales = np.array([
    [100, 200, 150],
    [250, 300, 200],
    [150, 100, 250]
```

])

Calculate average price of all products

average_price = np.mean(sales)

print("Average Price:", average_price)

3. **Scenario:** You are working on a project that involves analyzing a dataset containing information about houses in a neighborhood. The dataset is stored in a CSV file, and you have imported it into a NumPy array named `house_data`. Each row of the array represents a house, and the columns contain various features such as the number of bedrooms, square footage, and sale price.

Question: Using NumPy arrays and operations, how would you find the average sale price of houses with more than four bedrooms in the neighborhood?

```
import numpy as np

# Columns: Bedrooms, Square Feet, Sale Price
house_data = np.array([
    [3, 1200, 200000],
    [5, 1800, 350000],
    [4, 1500, 250000],
    [6, 2200, 450000],
    [2, 1000, 150000]
])

# Select houses with more than 4 bedrooms
filtered_houses = house_data[house_data[:, 0] > 4]

# Calculate average sale price
average_price = np.mean(filtered_houses[:, 2])

print("Average Sale Price:", average_price)
```

```
Average Sale Price: 400000.0
```

import numpy as np

Columns: Bedrooms, Square Feet, Sale Price

house_data = np.array([

[3, 1200, 200000],

[5, 1800, 350000],

[4, 1500, 250000],

[6, 2200, 450000],

[2, 1000, 150000]

```
])

# Select houses with more than 4 bedrooms
filtered_houses = house_data[house_data[:, 0] > 4]

# Calculate average sale price
average_price = np.mean(filtered_houses[:, 2])

print("Average Sale Price:", average_price)
```

4. **Scenario:** You are working on a project that involves analyzing the sales performance of a company over the past four quarters. The quarterly sales data is stored in a NumPy array named `sales_data`, where each element represents the sales amount for a specific quarter. Your task is to calculate the total sales for the year and determine the percentage increase in sales from the first quarter to the fourth quarter.

Question: Using NumPy arrays and arithmetic operations calculate the total sales for the year and determine the percentage increase in sales from the first quarter to the fourth quarter?

```
import numpy as np

# Sales for four quarters
sales_data = np.array([10000, 15000, 18000, 20000])

# Calculate total sales
total_sales = np.sum(sales_data)

# Calculate percentage increase
percentage_increase = ((sales_data[3] - sales_data[0])
                       / sales_data[0]) * 100

print("Total Sales:", total_sales)
print("Percentage Increase:", percentage_increase, "%")
```

main.py

```python
import numpy as np

# Sales for four quarters
sales_data = np.array([10000, 15000, 18000, 20000])

# Calculate total sales
total_sales = np.sum(sales_data)

# Calculate percentage increase
percentage_increase = ((sales_data[3] - sales_data[0])
                       / sales_data[0]) * 100

print("Total Sales:", total_sales)
print("Percentage Increase:", percentage_increase, "%")
```

```
Total Sales: 63000
Percentage Increase: 100.0 %
```

5. **Scenario:** You are a data analyst working for a car manufacturing company. As part of your analysis, you have a dataset containing information about the fuel efficiency of different car models. The dataset is stored in a NumPy array named `fuel_efficiency`, where each element represents the fuel efficiency (in miles per gallon) of a specific car model. Your task is to calculate the average fuel efficiency and determine the percentage improvement in fuel efficiency between two car models.

Question: How would you use NumPy arrays and arithmetic operations to calculate the average fuel efficiency and determine the percentage improvement in fuel efficiency between two car models?

```python
import numpy as np

# Fuel efficiency of car models in MPG
fuel_efficiency = np.array([20, 30, 40, 50])

# Calculate average fuel efficiency
average = np.mean(fuel_efficiency)

# Calculate percentage improvement between two models
improvement = ((fuel_efficiency[1] - fuel_efficiency[0])
               / fuel_efficiency[0]) * 100

print("Average Fuel Efficiency:", average, "MPG")
print("Percentage Improvement:", improvement, "%")
```

```
Average Fuel Efficiency: 35.0 MPG
Percentage Improvement: 50.0 %
```

```
import numpy as np

# Fuel efficiency of car models in MPG
fuel_efficiency = np.array([20, 30, 40, 50])

# Calculate average fuel efficiency
average = np.mean(fuel_efficiency)

# Calculate percentage improvement between two models
improvement = ((fuel_efficiency[1] - fuel_efficiency[0])
        / fuel_efficiency[0]) * 100

print("Average Fuel Efficiency:", average, "MPG")
print("Percentage Improvement:", improvement, "%")
```

6. **Scenario:** You are a cashier at a grocery store and need to calculate the total cost of a customer's purchase, including applicable discounts and taxes. You have the item prices and quantities in separate lists, and the discount and tax rates are given as percentages. Your task is to calculate the total cost for the customer.

Question: Use arithmetic operations to calculate the total cost of a customer's purchase, including discounts and taxes, given the item prices, quantities, discount rate, and tax rate?

```
# Item prices and quantities
prices = [100, 200, 150]
quantities = [2, 1, 3]

discount_rate = 10
tax_rate = 5

# Calculate total cost before discount
```

```
subtotal = sum(prices[i] * quantities[i] for i in range(3))

# Calculate discount
discount = subtotal * discount_rate / 100

# Calculate cost after discount
discounted_total = subtotal - discount

# Calculate tax
tax = discounted_total * tax_rate / 100

# Calculate final cost
total_cost = discounted_total + tax

print("Subtotal:", subtotal)
print("Discount:", discount)
print("Tax:", tax)
print("Final Total Cost:", total_cost)
```



7. **Scenario:** You are working as a data analyst for an e-commerce company. You have been given a dataset containing information about customer orders, stored in a Pandas DataFrame named `order_data`. The DataFrame has columns for customer ID, order date, product name, and order quantity. Your task is to analyze the data and answer specific questions about the orders.

Question: Using Pandas DataFrame operations, how would you find the following information from the `order_data` DataFrame:

1. The total number of orders made by each customer.
2. The average order quantity for each product.
3. The earliest and latest order dates in the dataset.

```
import pandas as pd


# Create a DataFrame
order_data = pd.DataFrame({
   'customer_id': [101, 102, 101, 103, 102],
   'order_date': ['2026-01-05', '2026-01-10',
                  '2026-02-15', '2026-02-20', '2026-03-01'],
   'product_name': ['Pen', 'Book', 'Pen', 'Bag', 'Book'],
   'quantity': [2, 3, 4, 1, 5]
})

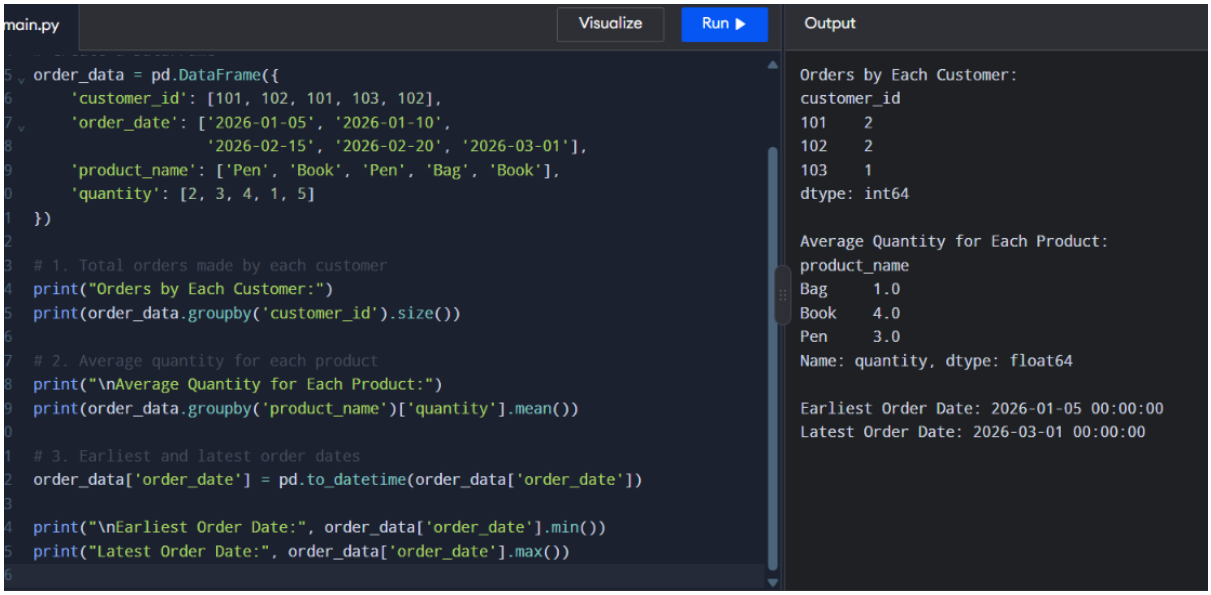

# 1. Total orders made by each customer
print("Orders by Each Customer:")
print(order_data.groupby('customer_id').size())


# 2. Average quantity for each product
print("\nAverage Quantity for Each Product:")
print(order_data.groupby('product_name')['quantity'].mean())


# 3. Earliest and latest order dates
order_data['order_date'] = pd.to_datetime(order_data['order_date'])
```

print("\nEarliest Order Date:", order_data['order_date'].min())

print("Latest Order Date:", order_data['order_date'].max())



8. **Scenario:** You are a data scientist working for a company that sells products online. You have been tasked with analyzing the sales data for the past month. The data is stored in a Pandas data frame.

Question: How would you find the top 5 products that have been sold the most in the past month?

import pandas as pd

Create a DataFrame

sales_data = pd.DataFrame({

'product_name': ['Pen', 'Book', 'Pen', 'Bag',

'Book', 'Pencil', 'Bag', 'Pen'],

'quantity_sold': [10, 20, 15, 8, 12, 25, 10, 30]

})

Calculate total sales quantity for each product

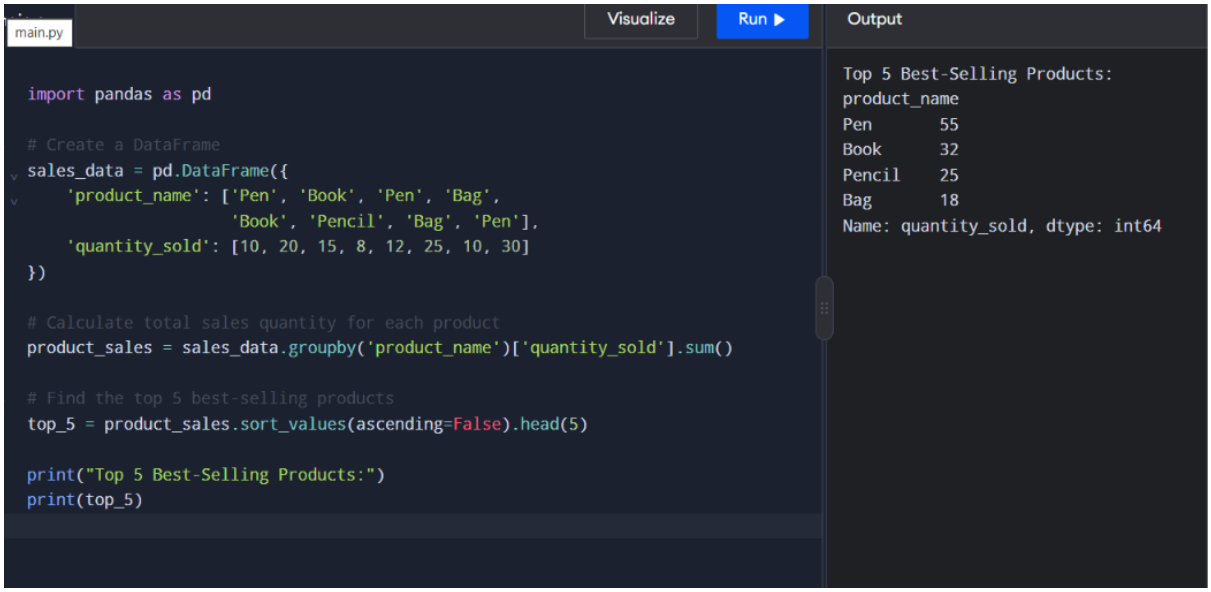
product_sales = sales_data.groupby('product_name')['quantity_sold'].sum()

Find the top 5 best-selling products

top_5 = product_sales.sort_values(ascending=False).head(5)

print("Top 5 Best-Selling Products:")

print(top_5)



9. **Scenario:** You work for a real estate agency and have been given a dataset containing information about properties for sale. The dataset is stored in a Pandas DataFrame named `property_data`. The DataFrame has columns for property ID, location, number of bedrooms, area in square feet, and listing price. Your task is to analyze the data and answer specific questions about the properties.

Question: Using Pandas DataFrame operations, how would you find the following information from the `property_data` DataFrame:

1. The average listing price of properties in each location.
2. The number of properties with more than four bedrooms.
3. The property with the largest area.

import pandas as pd

Create a DataFrame

property_data = pd.DataFrame({

'property_id': [101, 102, 103, 104, 105],

'location': ['Chennai', 'Vellore', 'Chennai',

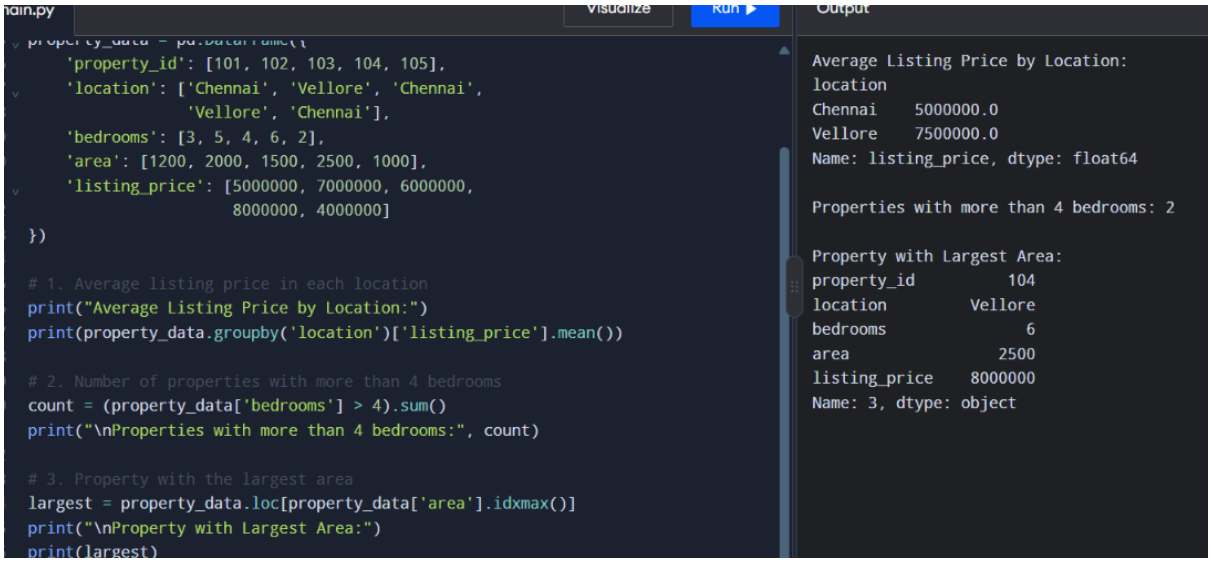
'Vellore', 'Chennai'],

```
    'bedrooms': [3, 5, 4, 6, 2],
    'area': [1200, 2000, 1500, 2500, 1000],
    'listing_price': [5000000, 7000000, 6000000,
                      8000000, 4000000]
})

# 1. Average listing price in each location
print("Average Listing Price by Location:")
print(property_data.groupby('location')['listing_price'].mean())

# 2. Number of properties with more than 4 bedrooms
count = (property_data['bedrooms'] > 4).sum()
print("\nProperties with more than 4 bedrooms:", count)

# 3. Property with the largest area
largest = property_data.loc[property_data['area'].idxmax()]
print("\nProperty with Largest Area:")
print(largest)
```



10. **Scenario:** You are working on a data visualization project and need to create basic plots using Matplotlib. You have a dataset containing the monthly sales data for a company, including the month and corresponding sales values. Your task is to develop a Python program that generates line plots and bar plots to visualize the sales data.

Question:

1. How would you develop a Python program to create a line plot of the monthly sales data?
2: How would you develop a Python program to create a bar plot of the monthly sales data?

```
import matplotlib.pyplot as plt


# Monthly sales data
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
sales = [1000, 1500, 1200, 1800, 2000, 2500]


# 1. Line plot
plt.plot(months, sales, marker='o')
plt.title("Monthly Sales - Line Plot")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.show()


# 2. Bar plot
plt.bar(months, sales)
plt.title("Monthly Sales - Bar Plot")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.show()
```